

# LAB REPORT: VISION-BASED TARGET TRACKING USING CAMERA

COURSE: MOBILE ROBOTICS LAB

---

**Platform:** TurtleBot3 Waffle (Gazebo Simulation)  
**Topic:** Image-based Perception and Closed-loop Control in ROS 2  
**Status:** Completed

## 1. OBJECTIVE & LEARNING OUTCOMES

The primary objective of this lab session was to implement an image-based perception and control loop using the onboard camera of the TurtleBot3 Waffle platform within a Gazebo simulation environment. The task involved subscribing to the `/camera/image_raw` topic, detecting a specific colored object, and translating that visual tracking information into real-time velocity commands to dynamically guide and align the robot.

By successfully completing this lab, the following key competencies were demonstrated:

- Subscription to and real-time processing of high-bandwidth image data strings within the ROS 2 ecosystem.
- Utilization of the OpenCV library and `cv_bridge` for image format conversions and color segmentation.
- Mathematical computation of the centroid of a target object to define structural error tracking metrics.
- Design and execution of a proportional control navigation law to achieve precise visual-servo docking behaviors.

## 2. SYSTEM ARCHITECTURE & PERCEPTION PIPELINE

The integration of sensory input to direct motor actions relies on a continuous, closed-loop processing pipeline. The data flow moves dynamically across ROS 2 nodes as follows:

`/camera/image_raw` → OpenCV Processing → Object Detection → Error Computation → `/cmd_vel`

The system begins by listening to raw frames published by the simulated camera. Using `cv_bridge`, these frames are translated from standard ROS `sensor_msgs/Image` formats into standard NumPy/OpenCV matrices for pixel manipulation and geometric segmentation.

### 3. IMPLEMENTATION DETAILS & CORE TASKS

The implementation was executed sequentially across five core experimental phases:

- **Task 1: Image Subscription and Visualization:** Established a robust subscriber node targeting `/camera/image_raw` and utilized `cv2.imshow()` to verify clean, live frame capture.
- **Task 2: HSV Color Segmentation:** Converted the captured RGB color frames into the Hue, Saturation, Value (HSV) color space to handle lighting variation. Binary mask thresholds were applied to cleanly isolate the target object from background noise.
- **Task 3: Centroid Extraction:** Calculated spatial image moments of the resulting binary mask to pinpoint the precise pixel coordinates  $(X_c, Y_c)$  of the object's center.
- **Task 4 & 5: Proportional Tracking and Docking:** Derived the error term as the horizontal delta between the image centerline and the target centroid. A proportional controller scaled this tracking error into an angular velocity command ( $\omega$ -axis) via a tuned gain ( $k_p$ ). Forward linear velocity ( $v$ -axis) was linked to target proximity, triggering a smooth stopping sequence at a pre-configured safe threshold distance.

### 4. EXPERIMENTAL DISCUSSION & EXERCISES

During the tuning parameters exercise (Exercise 1), variations in environmental lighting heavily influenced the stability of the HSV thresholding bounds. Widening bounds helped capture degraded targets but introduced false-positive pixel noise, emphasizing the need for fine-tuned masks or adaptive filtering.

For the motion controller validation (Exercise 2), selecting an excessively high proportional gain value ( $k_p$ ) led to rapid overcorrection, triggering violent physical oscillations about the target center. Conversely, configuring a small  $k_p$  resulted in a sluggish, heavily damped system response that often failed to converge quickly. The optimal gain was balanced to achieve snappy alignment with zero steady-state oscillation. Modifying the logic to zero-out angular velocities when the binary mask returned no tracking data (Exercise 3) successfully eliminated runaway rotations in the absence of an illuminated target.

## 5. CONCLUSION

This lab provided comprehensive, hands-on experience bridging raw visual perception and reactive motion control in ROS 2. By binding computer vision metrics directly to differential-drive kinematic actuation via `/cmd_vel`, a functional closed-loop visual-servoing node was fully realized. The methodology highlights how standard, low-level robotic platforms can leverage modern computer vision techniques to make self-governed navigation decisions in real-time.